A TRIMESTER – 1 CLASS PROJECT REPORT

EV CHARGING STATION SCHEDULING AND ENERGY OPTIMIZATION

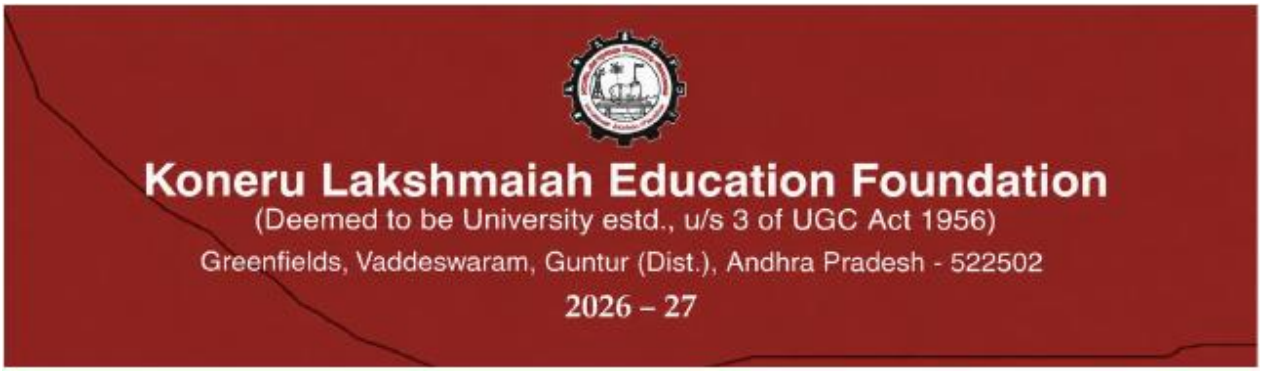
Using Greedy and Dynamic Programming Algorithms

Submitted in partial fulfillment of the requirements for
26CS5101E – Computational Thinking and Programming

Master of Technology in
Artificial Intelligence and Data Science

Submitted by
Vaishnavi Veda
Student ID: 2601150029

Under the supervision of
Dr. G. Syam Prasad
Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CERTIFICATE

This is to certify that the Trimester – 1 Class Project Report entitled "EV Charging Station Scheduling and Energy Optimization Using Greedy and Dynamic Programming Algorithms" is submitted by VAISHNAVI VEDA (Student ID: 2601150029) in partial fulfillment of the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science at K L University.

The project report describes the design and implementation of an EV charging scheduler that allocates charging requests to limited charging stations under time-window, energy, priority and cost constraints. The work is presented for the academic year 2026–2027.

Project Supervisor	Student
Dr. G. Syam Prasad Professor Signature: _____	VAISHNAVI VEDA 2601150029 Signature: _____

DECLARATION

I hereby declare that this project report entitled “EV Charging Station Scheduling and Energy Optimization Using Greedy and Dynamic Programming Algorithms” is a record of the project work prepared by me, VAISHNAVI VEDA (Student ID: 2601150029), in partial fulfillment of the requirements for 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science at K L University.

The report is based on the project files supplied for this work. Experimental outputs and test results are identified as observed from the included implementation; no claim is made that the prototype is a live charging network or a production deployment.

Place: _____

Date: _____

Signature: _____

VAISHNAVI VEDA

ACKNOWLEDGEMENT

I express my sincere gratitude to the management of K L University and the Department of Artificial Intelligence and Data Science for providing the academic environment and resources required to undertake this project.

I would like to thank my project supervisor, Dr. G. Syam Prasad, Professor, for his guidance and support throughout the project. His encouragement helped me connect computational thinking concepts with a practical scheduling problem.

I also acknowledge the documentation and learning resources for Python and algorithm design that supported the implementation. Finally, I thank everyone who contributed directly or indirectly to the completion of this project.

ABSTRACT

Electric vehicles (EVs) require charging facilities with limited charger capacity. When multiple vehicles arrive with different arrival times, departure deadlines, energy requirements and priority levels, assigning charging sessions becomes a scheduling problem. This project implements an EV Charging Station Scheduling module that compares two approaches: a greedy algorithm and dynamic programming based on weighted interval scheduling.

The system models charging requests and stations using Python data classes and validates important fields such as time windows, positive energy demand, station power and non-negative cost. The greedy scheduler processes urgent requests first and assigns each request to an earliest feasible charger slot. The dynamic-programming scheduler evaluates non-overlapping charging sessions and selects sessions according to priority-weighted energy value. The result reports scheduled sessions, rejected vehicle IDs, total energy delivered and total estimated cost.

The ZIP project also retains a Smart Parking System module demonstrating object-oriented programming, vehicle classes, parking-slot allocation, parking fees and mock payment processing. The EV scheduler is the primary focus of this report. A local demo run scheduled all six sample EV requests using both strategies and reported 240 kWh of total energy for each strategy; the greedy result cost 72.00 and the dynamic-programming result cost 82.00 for the included sample configuration. Three EV-specific automated tests passed when run locally. These observations apply only to the supplied demo inputs and do not establish universal algorithm superiority.

Keywords: Electric vehicles, charging scheduling, greedy algorithm, dynamic programming, weighted interval scheduling, Python, data classes, unit testing.

TABLE OF CONTENTS

S. No.	Title	Page
1	Introduction	7
2	Problem Statement and Requirements	8
3	Problem Solving and Computational Thinking	9
4	Python and Data Structures	10
5	Software Design and Engineering	11
6	Algorithms and Scheduling Logic	12
7	Software Quality and Testing	13
8	System Architecture and Implementation	14
9	Execution and Results	16
10	Complexity and Performance Discussion	17
11	Conclusion and Future Scope	18
12	References	19

1. INTRODUCTION

Electric vehicles are becoming an important part of modern transportation. Charging stations, however, have a limited number of chargers and each vehicle may need a different amount of energy before a specified departure time. A poor assignment can leave a charger idle, miss a vehicle's deadline or allocate resources to lower-priority requests while urgent requests remain unscheduled.

1.1 Project Overview

The EV Charging Station Scheduling module provides a small simulation for allocating EV charging requests across multiple stations. Each request includes a vehicle ID, arrival slot, departure slot, energy requirement and priority. Each station has an ID, a charging rate per time slot and a cost per slot. The scheduler returns a schedule with start and end slots, energy delivered, estimated cost and the algorithm used.

1.2 Objectives

- Model EV charging requests and charging stations with validated data structures.
- Implement and compare a greedy scheduling strategy and a dynamic-programming strategy.
- Respect arrival/departure windows and avoid overlapping sessions on the same station.
- Summarize delivered energy, total cost and rejected requests.
- Use automated tests to validate scheduling and input constraints.
- Retain the original smart parking subsystem as an additional demonstration of OOP concepts.

1.3 Scope

This is a command-line simulation based on discrete time slots. It does not connect to physical chargers, live vehicle telemetry, payment services or a production database. Cost values are simulated from station cost-per-slot settings.

1.4 Report Organization

The report explains the problem and requirements, computational thinking, data structures, design, algorithms, testing, observed output, limitations and future enhancements.

2. PROBLEM STATEMENT AND REQUIREMENTS

2.1 Problem Statement

Given a set of charging stations and EV requests, assign feasible charging sessions so that each accepted request receives its required energy within its arrival and departure window. Since station capacity is limited, sessions assigned to the same station must not overlap. The system should make scheduling decisions using clearly described strategies and report the resulting energy and cost.

2.2 Proposed Solution

The implementation defines EVRequest, ChargingStation and ChargingSchedule models. EVChargingScheduler exposes schedule_greedy() and schedule_dynamic_programming(). Both methods return a ScheduleResult containing the selected sessions, rejected vehicle IDs, total energy and total cost.

2.3 Functional Requirements

ID	Requirement	Description
FR01	Represent EV requests	Store vehicle ID, arrival slot, departure slot, energy requirement and priority.
FR02	Represent stations	Store station ID, charging power per slot and cost per slot.
FR03	Greedy scheduling	Order requests by urgency and assign a feasible station/time interval.
FR04	DP scheduling	Use weighted interval scheduling to select non-overlapping candidate sessions.
FR05	Summarize results	Return schedules, rejected IDs, total kWh and total cost.
FR06	Validate input	Reject invalid time windows, non-positive energy or priority, invalid station power and negative cost.
FR07	Demonstrate execution	Run a fixed example through both algorithms and print results.

2.4 Non-Functional Requirements

- Readability through small modules and descriptive names.
- Predictable output for fixed demo inputs.
- Input validation at model creation.
- Repeatable verification through automated tests.

- Clear separation between data models, scheduling logic and demo presentation.

3. PROBLEM SOLVING AND COMPUTATIONAL THINKING

3.1 Decomposition

The problem is divided into model definition, input validation, duration calculation, overlap checking, greedy assignment, dynamic-programming selection, result aggregation and output display. Each responsibility is implemented in a focused function or class.

3.2 Abstraction

A vehicle is represented by an EVRequest rather than by a full physical vehicle model. A station is represented by its ID, power per slot and cost per slot. The scheduler works with these essential values and does not model battery chemistry or real charger communication.

3.3 Pattern Recognition

Every request follows the same field structure. Charging duration can be calculated consistently from energy demand and station power. Time intervals can be checked using a shared overlap rule. These repeated patterns make the logic reusable.

3.4 Algorithmic Thinking

The greedy approach sorts requests using departure time, priority, energy demand and arrival time, then searches stations and feasible start slots. The DP approach creates candidate intervals and uses the weighted interval scheduling recurrence to compare including or excluding each candidate.

3.5 Complexity Awareness

Operation	Approach	Indicative cost
Request ordering	Sort requests by scheduling keys	O(r log r)
Overlap check	Compare candidate interval with occupied intervals	O(k) per check, where k is occupied intervals
Greedy slot search	Try feasible start slots across stations	Depends on requests, stations and window length
DP predecessor search	Find previous non-overlapping candidate	$O(c^2)$ in this implementation
DP recurrence	Compute best value for sorted candidates	O(c) after predecessor calculation
Result summary	Sum energy and cost over scheduled sessions	O(s)

Here, r is the number of requests, c is the number of DP candidate sessions for a station, s is the number of scheduled sessions, and k is the number of intervals already assigned to a station. These are explanatory bounds for the implemented approach, not measured runtime guarantees.

4. PYTHON AND DATA STRUCTURES

4.1 Python Implementation

Python is used for the EV data models, scheduling algorithms and command-line demonstration. The code uses type annotations, data classes, list and dictionary collections, sorting, loops and exception-based validation.

4.2 Core Data Models

Class	Important fields	Purpose
EVRequest	vehicle_id, arrival_slot, departure_slot, energy_needed_kwh, priority	Describes a vehicle's charging requirement and time window.
ChargingStation	station_id, power_kwh_per_slot, cost_per_slot	Describes station capacity and simulated slot cost.
ChargingSchedule	vehicle_id, station_id, start_slot, end_slot, energy_delivered_kwh, cost, algorithm	Stores one scheduled charging session.
ScheduleResult	schedules, rejected_vehicle_ids, total_energy_kwh, total_cost	Collects the complete result of a scheduling run.

4.3 Data Structures Used

Structure	Use	Reason
List	Requests, stations, schedules and rejected IDs	Maintains ordered collections and supports iteration.
Dictionary	Greedy station occupancy keyed by station ID	Provides direct access to each station's occupied intervals.
Tuple	Time intervals and internal candidate records	Groups related values compactly.
Set	Chosen vehicle IDs in DP	Prevents a request selected for one station from being selected again for another station.
Data class	EVRequest, ChargingStation, ChargingSchedule, ScheduleResult	Keeps related fields explicit and readable.

4.4 Validation

EVRequest rejects negative arrival times, a departure time that is not later than arrival, non-positive energy and non-positive priority. ChargingStation rejects non-positive charging power and negative cost. These checks prevent invalid values from entering the scheduling logic.

5. SOFTWARE DESIGN AND ENGINEERING

5.1 Modular Design

The EV scheduler is organized in the ev_charging package. models.py contains the data classes, scheduler.py contains scheduling and result logic, and ev_charging_demo.py provides the runnable example. tests/test_ev_charging.py checks representative behavior.

5.2 Object-Oriented Design

EVChargingScheduler encapsulates the available stations and exposes two scheduling methods. Data classes represent the domain entities and make inputs and outputs explicit. The original project also contains a Smart Parking System with vehicle subclasses, parking spots, a parking lot and payment handling.

5.3 Architecture

Layer	Module / component	Responsibility
Data model	ev_charging/models.py	Define and validate requests, stations and schedule records.
Scheduling logic	ev_charging/scheduler.py	Calculate charging duration, detect interval conflicts and implement both strategies.
Demo / presentation	ev_charging_demo.py	Create sample inputs and print schedule summaries.
Verification	tests/test_ev_charging.py	Check greedy output, DP output and invalid request validation.
Supporting subsystem	parking_system.py, vehicles.py, payment_gateway.py and services/	Retain the original parking simulation and mock payment functionality.

5.4 API and Interface

The EV module is a command-line program and does not expose a web API or graphical interface in the supplied implementation. The demo prints each assigned vehicle, station, slot range, energy and cost, followed by totals and rejected IDs.

5.5 Error Handling

The model constructors raise ValueError for invalid values. The scheduler constructor raises ValueError if no charging stations are supplied. These checks make common invalid configurations visible rather than silently accepting them.

6. ALGORITHMS AND SCHEDULING LOGIC

6.1 Charging Duration

The required number of time slots is calculated by rounding up the energy demand divided by the station's charging power per slot:

duration = ceil(energy_needed_kwh / power_kwh_per_slot)

Rounding up ensures that a request requiring part of an additional slot is allocated a whole slot in this discrete model.

6.2 Interval Overlap

Two intervals overlap when the candidate start is earlier than an occupied interval's end and the candidate end is later than its start. Adjacent intervals, such as [0, 2) and [2, 4), do not overlap.

6.3 Greedy Algorithm

1. Convert the request iterable into a list.
2. Sort requests by earliest departure, higher priority, lower energy demand and earlier arrival.
3. For each request, calculate its duration on each station.
4. Search start slots from arrival through the latest feasible start.
5. Select the earliest-ending feasible candidate, with cost and start time used to break ties.
6. Reserve the chosen interval or add the vehicle ID to the rejected list if no feasible assignment exists.
7. Return sorted schedules and aggregate energy and cost.

6.4 Dynamic Programming Algorithm

The DP method applies weighted interval scheduling separately to stations ordered by lower cost and higher charging power. For each remaining request, it creates a candidate session at the request's arrival slot if the session finishes by the departure deadline. The candidate value is priority multiplied by energy needed.

DP[i] = max(value[i] + DP[p(i) + 1], DP[i - 1])

Here, p(i) identifies the last candidate that finishes no later than candidate i begins. The recurrence compares selecting the current candidate with skipping it. Chosen vehicle IDs are removed before processing the next station so that the same vehicle is not assigned twice.

6.5 Algorithmic Limitation

The DP implementation generates one earliest-start candidate per request and station and processes stations sequentially. It is therefore a simplified scheduling heuristic built around weighted interval scheduling, not a global optimizer over every possible start time and every station combination. The greedy and DP outputs should be compared on the same input, and the objective being optimized should be stated clearly.

7. SOFTWARE QUALITY AND TESTING

7.1 Testing Approach

The EV-specific test module uses pytest. Tests verify that greedy scheduling creates a schedule and delivers positive energy, that DP schedules requests without duplicate vehicle assignments, and that an invalid request raises ValueError.

7.2 Test Cases

Test	Input / action	Expected behavior	Observed status
Greedy scheduling	Use two stations and three valid requests	At least one session is scheduled and total energy is positive	Passed
DP scheduling	Use the same sample data	Sessions exist and vehicle IDs are unique	Passed
Invalid request	Departure slot earlier than arrival slot	ValueError is raised	Passed

7.3 Test Execution

Command used in the local project folder:

python -m pytest -q tests/test_ev_charging.py

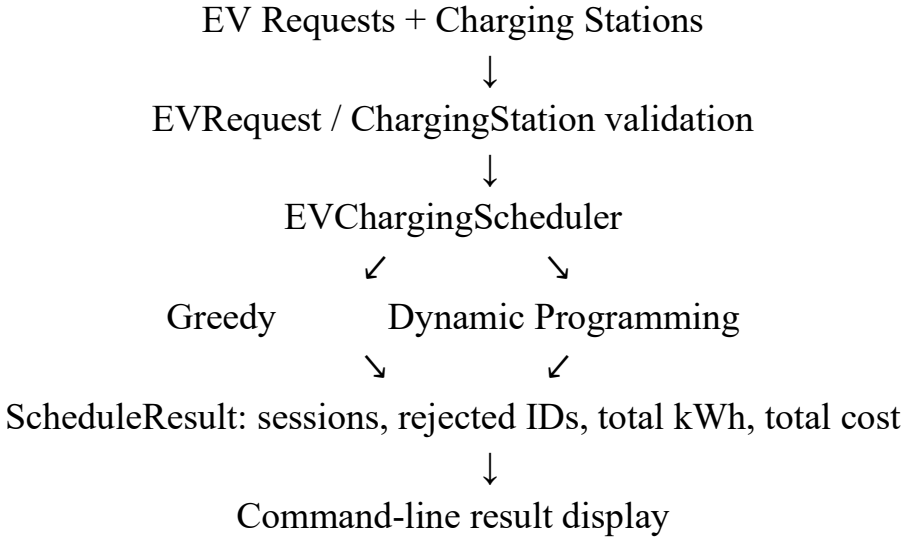
Observed result: 3 passed. This status refers only to the EV-specific test file and the local environment used for this report. It should not be interpreted as proof that every test in the entire ZIP project passes.

7.4 Quality Practices

- Keep models and algorithms in separate modules.
- Use explicit type annotations and descriptive identifiers.
- Validate invalid inputs at model construction.
- Use deterministic demo inputs for repeatable comparison.
- Run tests again after any changes before submitting the final report.

8. SYSTEM ARCHITECTURE AND IMPLEMENTATION

8.1 High-Level Flow



8.2 Demo Configuration

Station ID	Power per slot	Cost per slot
CS-01	20 kWh	6.00
CS-02	30 kWh	8.00
CS-03	15 kWh	4.00

Vehicle ID	Arrival	Departure	Energy needed	Priority
EV-101	0	4	40 kWh	3
EV-102	0	3	30 kWh	2
EV-103	1	5	50 kWh	4
EV-104	2	6	35 kWh	1
EV-105	3	7	60 kWh	5
EV-106	4	8	25 kWh	2

8.3 Running the Project

From the extracted EV-Charging-Station-Scheduling project directory, run:

python ev_charging_demo.py

To run the EV-specific tests, use:

```
python -m pytest -q tests/test_ev_charging.py
```

8.4 Main Files

File	Role
ev_charging/models.py	Validated data classes for requests, stations and schedules.
ev_charging/scheduler.py	Greedy and dynamic-programming scheduling logic.
ev_charging_demo.py	Sample configuration and printed output.
tests/test_ev_charging.py	Three EV scheduler tests.
EV_PROJECT_README.md	Overview of the EV extension and its algorithms.
main.py and parking modules	Original smart parking simulation retained in the ZIP.

9. EXECUTION AND RESULTS

9.1 Greedy Schedule Output

Vehicle	Station	Slot range	Energy	Cost
EV-101	CS-01	0–2	40 kWh	12.00
EV-102	CS-02	0–1	30 kWh	8.00
EV-103	CS-02	1–3	50 kWh	16.00
EV-104	CS-01	2–4	35 kWh	12.00
EV-105	CS-02	3–5	60 kWh	16.00
EV-106	CS-03	4–6	25 kWh	8.00
Total	—	—	240 kWh	72.00

Observed from the included demo: all six requests were scheduled by the greedy strategy and no vehicle IDs were rejected.

9.2 Dynamic-Programming Schedule Output

Vehicle	Station	Slot range	Energy	Cost
EV-102	CS-02	0–1	30 kWh	8.00
EV-101	CS-03	0–3	40 kWh	12.00
EV-103	CS-01	1–4	50 kWh	18.00
EV-104	CS-02	2–4	35 kWh	16.00
EV-105	CS-03	3–7	60 kWh	16.00
EV-106	CS-01	4–6	25 kWh	12.00
Total	—	—	240 kWh	82.00

Observed from the included demo: all six requests were scheduled by the DP strategy and no vehicle IDs were rejected.

9.3 Comparison

Measure	Greedy	Dynamic Programming
Requests scheduled	6 of 6	6 of 6
Rejected requests	0	0
Total energy	240 kWh	240 kWh
Total simulated cost	72.00	82.00
Selection principle	Urgency-first feasible assignment	Priority-weighted interval selection by station

For this particular sample, the greedy strategy delivered the same total energy at a lower simulated cost. The DP method's selection value is priority × energy, not minimum total cost, so a higher cost is consistent with its stated objective. This one demo is not enough to conclude that greedy scheduling is always cheaper or that DP is always better at prioritizing urgent vehicles.

9.4 Result Summary

The demo confirms that both implemented methods produce schedule records and aggregate energy/cost summaries for the supplied example. The automated EV-specific test run completed with three passing tests.

10. COMPLEXITY AND PERFORMANCE DISCUSSION

10.1 Performance Considerations

The greedy method sorts requests and searches feasible time slots on stations. Its runtime depends on the number of requests, number of stations, length of the allowed time windows and the number of occupied intervals checked. The DP method sorts candidate sessions and finds compatible predecessors using nested loops, which makes the predecessor calculation quadratic in the number of candidates for each station.

10.2 What Was Measured

The local verification recorded in this report consists of successful execution of ev_charging_demo.py and a test run of tests/test_ev_charging.py with three passing tests. No controlled timing benchmark between the greedy and DP EV algorithms was included in the supplied performance output, so this report does not claim a measured speed advantage for either algorithm.

10.3 Interpretation

A meaningful performance study should vary request count, station count, time-window length, energy demands and priority distributions. Each configuration should be run repeatedly using the same input for both algorithms. Runtime, scheduled requests, total delivered energy, total cost and rejected requests should be recorded together because speed alone does not measure scheduling quality.

10.4 Limitations

- The time axis is represented by integer slots rather than real timestamps.
- The model does not include battery state, charging curves, grid load or renewable energy availability.
- The DP method evaluates a limited candidate set and processes stations sequentially.
- Cost is a simple station cost-per-slot calculation and is not a real tariff or billing integration.
- The demo is a simulation; no live charger hardware or external service is connected.

11. CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

The project demonstrates a Python-based simulation for scheduling EV charging requests across multiple stations. It models charging requests and station properties, validates inputs, checks time-window feasibility and compares greedy assignment with a dynamic-programming method based on weighted interval scheduling. For the supplied demo, both algorithms scheduled six vehicles and delivered 240 kWh; the greedy result reported a total cost of 72.00 and the DP result 82.00. Three EV-specific automated tests passed in the local run.

The results show how algorithm choice reflects the objective being optimized. The greedy strategy prioritizes early departure and feasible allocation, while the DP strategy uses priority-weighted energy as its candidate value. More comprehensive experiments are required before drawing general conclusions about quality or performance.

11.2 Learning Outcomes

- Representing a practical problem through data models and constraints.
- Applying sorting, interval overlap checks and weighted interval scheduling.
- Using lists, dictionaries, tuples and sets to manage scheduling state.
- Separating domain models, algorithm logic, demo code and tests.
- Interpreting observed results without overgeneralizing from a single example.

11.3 Future Enhancements

- Use real timestamps, time-dependent tariffs and variable charging rates.
- Optimize across all stations and candidate start times jointly.
- Add fairness, waiting time, departure urgency and grid-load objectives.
- Build a web dashboard for station status and schedule visualization.
- Persist requests and schedules in a database and add user authentication.
- Connect to simulated or real charger telemetry through a well-defined API.
- Create larger benchmark datasets and report repeated runtime and quality measurements.

11.4 Additional Scope from the ZIP

The ZIP also includes a Smart Parking System that models bikes, cars, SUVs and trucks; assigns vehicles to differently sized spots; calculates parking fees; and

simulates cash, card or UPI payment. This subsystem can be developed further or integrated as a separate mobility-facility module, but it is distinct from the EV charging scheduling algorithm described in the main chapters.

12. REFERENCES

[1] Python Software Foundation, “Python Documentation.” https://docs.python.org/3/

[2] Python Software Foundation, “dataclasses — Data Classes.” https://docs.python.org/3/library/dataclasses.html

[3] pytest, “pytest Documentation.” https://docs.pytest.org/

[4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., Introduction to Algorithms, 3rd ed., MIT Press, 2009. (Dynamic programming and weighted interval scheduling concepts.)

[5] Kleinberg, J. and Tardos, É., Algorithm Design, Pearson, 2005. (Greedy algorithms and dynamic programming.)

[6] EV-Charging-Station-Scheduling project files supplied for this report: EV_PROJECT_README.md, ev_charging/models.py, ev_charging/scheduler.py, ev_charging_demo.py and tests/test_ev_charging.py.

[7] Smart Parking System project documentation supplied in README.md and notes.md within the same ZIP archive.